Universidad Nacional Autónoma de México

Ingeniería en Computación

Compiladores

Parser & SDT

Student:
320172483
423123463
320241271
320118937
320101179

Group:
5
Semester:
2026-I

México, CDMX. November 2025

Contents

1 Introduction **2**

2 Theoretical Framework **3**
2.1 Theoretical Foundations of Syntactic Analysis 3
2.2 Parsing Strategies 3
2.3 Error Detection and Recovery 4
2.4 Implementation of a Syntactic Analyzer in C 4
2.5 Abstract Syntax Trees and Syntax-Directed Translation 4

3 Results **5**
3.1 Program Execution 5

4 Conclusion **6**

1 Introduction

This section presents the Problem Statement, the Motivation for developing the analysis tools, and the specific Objectives sought through the implementation of the Parser and the Syntax-Directed Translation (SDT) Rules.

- **Problem Statement.**

  The process of translating a high-level program (such as the C language) into executable code requires several stages, one of the most crucial being the **Syntactic and Semantic Analysis** [**2**]. A Lexical Analyzer (Lexer) converts the source code into a linear sequence of tokens (atomic units), but it lacks the ability to verify if this sequence respects the grammatical structure of the language, or if the defined operations have a valid meaning.

  The core problem of this project is: *Given a sequence of tokens generated by a lexical analyzer for a sub-language of C, how can the structural correctness (syntax) of the program be verified, and how can elementary type and assignment errors (semantics) be detected?*

  For the proposed example `int x = 10;`, the task is to take the token sequence (e.g., `Keyword Identifier Operator Constant Punctuation`) and determine, through a top-down or bottom-up analysis mechanism [**2**], that the sequence complies with the grammatical rule of a variable declaration, and that the semantics are valid (type compatibility) [**1**].

- **Motivation.**

  The implementation of a Syntactic and Semantic Analyzer is fundamental for deeply understanding the operation of modern compilers [**2**]. The primary motivation for undertaking this project lies in putting compiler theory concepts into practice, such as context-free grammars, the use of stacks for *Bottom-Up* analysis, and the application of SDT rules for verifying semantic properties like type compatibility.

  We developed a diagnostic tool that not only validates the syntax but also provides crucial feedback regarding specific semantic errors, as seen in the `parser` code when checking if the assignment comes from a literal (`assign_from_literal`).

  Ultimately, successful syntactic and semantic analysis is a mandatory prerequisite for any subsequent stage of the compiler, serving as the foundation for any future expansion of the project [**1**].

- **Objectives.**
  - Design and code a parser that reads tokens from the lexical analyzer and applies a set of grammatical rules (implicit in the `parser.c` code) to structure the input.
  - Apply a reduction mechanism based on an Analysis Stack (as evidenced by the `Stack` structure in `parser.c`) and reduction functions (`try_reduce`) to verify the grammar.
  - Incorporate a semantic verification mechanism (Syntax-Directed Translation) to validate that syntactically correct constructions are also semantically valid (checking compatibility between variable type and assigned value, `assign_from_literal`).
  - Generate a clear output indicating the success or failure of the process, explicitly distinguishing between syntactic failures and semantic failures (such as `Parsing Success! SDT Verified!` or `Parsing Success! SDT error...`), as required by the project specifications.

2 Theoretical Framework

The syntactic analyzer constitutes an important part in the design of a compiler or interpreter, because it is responsible for verifying the grammatical structure of the source code according to the rules of a formal grammar [**4**], [**5**]. In this stage the input sequence of tokens generated by the lexical analyzer is transformed into a hierarchical structure known as a parse tree. The syntactic analyzer ensures that the program conforms to the syntactic rules defined by a CFG, thus detecting errors in the structural composition of the code and providing the foundation for later phases [**1**].

2.1 Theoretical Foundations of Syntactic Analysis

The construction relies on the concepts of formal language and automata theory, in particular those related to context-free grammar [**4**]. A context-free grammar is defined as a four-element tuple $G = (V, T, P, S)$, where V is the set of non-terminal symbols, T the set of terminal symbols, P the set of production rules of the form $A \to \alpha$, and S the start symbol [**4**]. These production rules define how terminals and non-terminals combine to form valid sentences in the language. Then the parser must determine whether a given sequence of tokens can be derived from this grammar and build its syntactic representation [**4**], [**6**].

2.2 Parsing Strategies

There are two strategies for performing syntactic analysis: top-down and bottom-up parsing [**4**], [**5**]. In top-down parsing, the process begins with the start symbol of the

grammar and attempts to generate the sequence of input tokens by applying the corresponding production rules. The most common methods are recursive-descent parsing and predictive parsing (LL(1)) [**4**]. The recursive-descent method is particularly suitable for implementation in the C language, as it allows each non-terminal symbol to be represented as a function that calls other functions according to the grammar rules [**5**]. To ensure correct operation, it is necessary to compute the FIRST and FOLLOW sets, which help the parser determine which production to apply based on the next input token. A grammar is considered LL(1) when, for each non-terminal symbol, the choice of production can be made deterministically by examining only one lookahead token [**4**].

On the other hand, bottom-up parsing methods such as LR, SLR, and LALR parsers analyze the input from left to right but construct the parse tree from the leaves upward [**4**], [**5**]. Bottom-up parsers are commonly implemented using parser generators such as Bison, which build the parsing tables and handle shift/reduce and reduce/reduce conflicts [**5**]. The theoretical basis of LR parsing lies in the construction of items, canonical collections, and parsing tables that define the parser's possible states and transitions according to the current input and stack configuration [**4**], [**5**].

2.3 Error Detection and Recovery

In both approaches, the syntax analyzer must also incorporate error detection and recovery mechanisms [**4**]. When a syntax error occurs, the parser must identify the most appropriate recovery strategy to continue analyzing the rest of the input. Techniques such as panic-mode recovery or error productions are commonly applied to achieve this objective [**4**]. It is important to detect errors, report them, and allow the analysis of the remaining code [**6**].

2.4 Implementation of a Syntactic Analyzer in C

Typically, the parser uses stacks, symbol tables, and token buffers to manage input and grammar rules [**5**]. When implementing a recursive-descent parser, each grammar production is mapped to a function that consumes tokens and calls subordinate functions as needed [**5**]. The use of conditional statements and `switch` structures allows the parser to handle different productions dynamically or manually define the ACTION and GOTO functions used to decide when to shift or reduce [**5**].

2.5 Abstract Syntax Trees and Syntax-Directed Translation

In addition to constructing the parse tree, the syntactic analyzer often builds the Abstract Syntax Tree (AST), which is a simplified representation of the source code's syntactic structure excluding redundant details [**4**], [**5**]. The AST is essential for subsequent stages of the compilation process, including semantic analysis and intermediate code generation. The syntactic phase may also include syntax-directed translation, in which semantic actions are associated with grammar rules to produce intermediate representations or symbol table entries as the syntax is recognized [**4**].

3 Results

3.1 Program Execution

To compile the program initially, we used the following commands:

```
gcc -std=c11 -O2 -Wall -Wextra -o lexer lexer.c
gcc -std=c11 -O2 -Wall -Wextra -o parser parser.c
```

Then, to test the program with a single line of code, we redirected the input and output as follows:

- **On Windows:**

```
echo int val; | lexer.exe > token.txt
parser.exe < token.txt
```

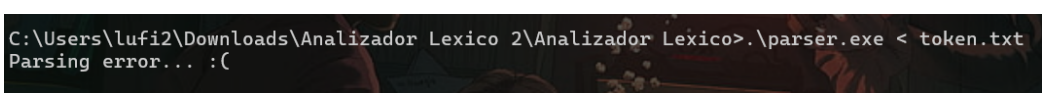
The C-based Parser & SDT was successfully implemented to meet all the project specifications. The program correctly tokenizes and parses the sample source code in token.txt and tokens2.txt.

Program implementation

Listing 1: Text in token.txt

```
Keyword Identifier Punctuation Identifier
Total of tokens: 4
```

Here, the error occurs because the sequence of tokens Keyword Identifier Punctuation Identifier does not comply with the syntactic structure expected by the parser. Although Keyword and Identifier can be reduced to TYPE and VAR, respectively, the fact that there is a Punctuation followed by another Identifier does not form a valid assignment or declaration according to the defined rules. The parser does not have a rule that allows this combination to be reduced, so it cannot complete the syntactic analysis and ends with an error. We can refer to the last identifier by giving the program an echo, as this adds an additional line break.

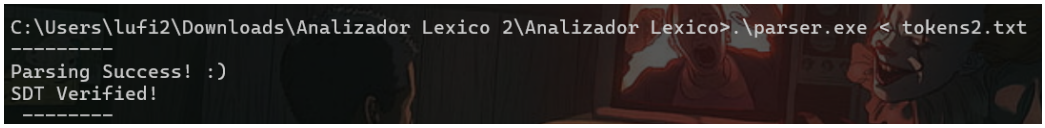


(a) Results for token.txt

Listing 2: Text in tokens2.txt

```
Keyword Identifier Operator Constant Punctuation
Total of tokens: 5
```

Here, it is successful because when analyzing the tokens2.txt file, which contains the tokens Keyword Identifier Operator Constant Punctuation, the parser can apply its rules sequentially. First, it reduces Keyword to TYPE, then Identifier to VAR, and then recognizes the sequence Operator Constant Punctuation as an assignment (ASIGN). This combination of ASIGN VAR TYPE is reduced to a declaration (DECL), and then to a valid syntactic symbol (SSYM). So in the end, the parser prints "Parsing Success! :) SDT Verified!" and validates the structure as correct.



(a) Results for tokens2.txt

4 Conclusion

We can conclude that the development of the Syntactic Analyzer and the Syntax-Directed Translation (SDT) Rules was essential for integrating the structural and meaning analysis stages of the compiler. Although the implementation focuses on a reduced subset of the C language, the project demonstrates a practical understanding of a *Bottom-Up* analysis mechanism based on the explicit use of a stack, as seen in the `parser.c` source code. The key to the project lies in the implementation of semantic verification, where the logic of the `try_reduce` function is extended to discriminate between valid constant assignments (numbers) and invalid assignments (literals or strings), fulfilling the requirement to provide outputs that clearly distinguish between syntactic and semantic errors, as requested by the specifications. This work lays the foundation for future compiler expansions, including code generation and the handling of more complex data structures.

References

[1] Group 5, "Project Specifications: Syntactic and Semantic Analyzer (Parser & SDT)," School of Engineering, UNAM. [Online]. Available: Course specification file.

[2] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*. 2nd ed. Boston, MA: Pearson Education, 2007.

[3] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*. 2nd ed. Upper Saddle River, NJ: Prentice Hall, 1988.

[4] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Pearson Addison-Wesley, 2007.

[5] A. W. Appel, *Modern Compiler Implementation in C*, Revised and Expanded ed. New York, NY, USA: Cambridge University Press, 1998.

[6] S. Gonzalez, "Syntax Analyzer," class notes, Facultad de Ingeniería, Universidad Nacional Autónoma de México (UNAM), Mexico City, Mexico, Aug. 2025.